

Error Handling & Debugging — Error মোকাবিলা ও Debugging

এখন আমরা JavaScript-এর খুব গুরুত্বপূর্ণ একটি অংশ শিখব: **Error Handling এবং Debugging**। বাস্তব জীবনের application-এ user ভুল input দিতে পারে, API fail করতে পারে, কোনো variable ভুল হতে পারে, server response না-ও আসতে পারে। তখন program যেন পুরোপুরি crash না করে এবং developer যেন সমস্যাটা সহজে খুঁজে বের করতে পারে—এই কাজের জন্য Error Handling ও Debugging ব্যবহার করা হয়।

Error Detection → Graceful Handling (try...catch) → Clean Recovery (finally) → Root-Cause Debugging

নির্ভুল ও ক্রাশ-মুক্ত প্রফেশনাল ওয়েব অ্যাপ্লিকেশন তৈরির জন্য এই অধ্যায়টি অপরিহার্য! 🔥

PART 01 — JAVASCRIPT ERROR ARCHITECTURE & ERROR TYPES (14.1 – 14.10)

14.1 Error কী? — Error কী জিনিস?

Error হলো program-এর এমন একটি সমস্যা যার কারণে JavaScript প্রত্যাশিতভাবে কাজ করতে পারে না।

💡 REAL-LIFE EXAMPLE

ধরো তুমি একটি ATM-এ গেলে:

- Account number ভুল → সমস্যা
- PIN ভুল → সমস্যা
- Balance কম → সমস্যা
- ATM server unavailable → সমস্যা

Programming-এও একই ধরনের সমস্যা হয়।

14.1 JAVASCRIPT

RUN

```
let age = 20;  
  
console.log(age);
```

এখানে কোনো সমস্যা নেই।

কিন্তু:

14.1 JAVASCRIPT

RUN

```
console.log(username);
```

এখানে `username` declare করা হয়নি।

```
ReferenceError: username is not defined
```

14.2 Error বনাম Exception

দুটো শব্দ অনেক সময় একই অর্থে ব্যবহার করা হলেও conceptually:

- **Error** → program-এর সমস্যা
- **Exception** → runtime-এ এমন একটি exceptional condition যা আমরা handle করতে পারি

উদাহরণ:

14.2 JAVASCRIPT

RUN

```
let result = 10 / 0;  
  
console.log(result);
```

TEXT

Infinity

এটা error নয়।

কিন্তু:

14.2 JAVASCRIPT

RUN

```
throw new Error("Something went wrong");
```

এখানে আমরা ইচ্ছাকৃতভাবে একটি exception throw করেছি।

14.3 Common JavaScript Errors

JavaScript-এ কয়েক ধরনের গুরুত্বপূর্ণ built-in error আছে।

Error	Meaning
<code>SyntaxError</code>	Code-এর syntax ভুল
<code>ReferenceError</code>	এমন variable/reference ব্যবহার করা হয়েছে যা নেই
<code>TypeError</code>	ভুল type-এর operation
<code>RangeError</code>	Valid range-এর বাইরে value
<code>URIError</code>	URI-related ভুল
<code>EvalError</code>	<code>eval()</code> -related error; modern code-এ খুব কম ব্যবহৃত
<code>AggregateError</code>	একাধিক error একসাথে represent করতে ব্যবহৃত

14.4 SyntaxError

Syntax ভুল হলে **SyntaxError** হয়।

14.4 JAVASCRIPT

RUN

```
if (true {  
  console.log("Hello");  
}
```

SyntaxError

কারণ:

14.4 JAVASCRIPT

RUN

```
if (true)
```

এর পরে `)` missing।

সঠিক:

14.4 JAVASCRIPT

RUN

```
if (true) {  
  console.log("Hello");  
}
```

TEXT

Hello

সহজভাবে

Compiler/JavaScript engine বলতে পারে:

"তোমার code-এর grammar ঠিক নেই।"

14.5 ReferenceError

যে variable exist করে না সেটি access করলে **ReferenceError** হয়।

14.5 JAVASCRIPT

RUN

```
console.log(userName);
```

ReferenceError: userName is not defined

সঠিক:

14.5 JAVASCRIPT

RUN

```
let userName = "Shariar";  
  
console.log(userName);
```

TEXT

Shariar

💡 REAL-LIFE EXAMPLE

তুমি বললে:

"আমাকে Rahim-এর phone number দাও।"

কিন্তু তোমার contact list-এ Rahim-এর কোনো record নেই।

Programming-এ একইভাবে unknown variable → **ReferenceError** ।

14.6 TypeError

ভুল data type-এর ওপর এমন operation করলে **TypeError** হতে পারে।

14.6 JAVASCRIPT

RUN

```
let user = null;  
  
console.log(user.name);
```

TypeError

কারণ `null`-এর মধ্যে `name` access করা যাচ্ছে না।

আরেকটি example:

14.6 JAVASCRIPT

RUN

```
let age = 20;  
  
age();
```

TypeError: age is not a function

কারণ `age` একটি number, function নয়।

14.7 RangeError

কোনো value valid range-এর বাইরে গেলে **RangeError** হতে পারে।

14.7 JAVASCRIPT

RUN

```
let number = 10;  
  
console.log(number.toFixed(101));
```

এখানে decimal digits-এর valid range-এর বাইরে value দেওয়া হয়েছে।

RangeError

আরেকটি example:

14.7 JAVASCRIPT

RUN

```
let arr = new Array(-5);
```

RangeError

কারণ array length negative হতে পারে না।

14.8 URIError

URI-related function-এ invalid URI দিলে **URIError** হতে পারে।

14.8 JAVASCRIPT

RUN

```
decodeURIComponent("%");
```

URIError

কারণ `%` এখানে valid encoded URI sequence নয়।

14.9 Error Object

JavaScript-এ আমরা `Error` object তৈরি করতে পারি।

14.9 JAVASCRIPT

RUN

```
let error = new Error("Something went wrong");  
  
console.log(error);
```

Error: Something went wrong

14.10 Error-এর গুরুত্বপূর্ণ Properties

একটি Error object-এর গুরুত্বপূর্ণ properties:

14.10 JAVASCRIPT

RUN

```
let error = new Error("Invalid username");  
  
console.log(error.name);  
console.log(error.message);  
console.log(error.stack);
```

Output-এর structure হবে এরকম:

```
Error  
Invalid username  
Error: Invalid username  
    at ...
```

★ তিনটি গুরুত্বপূর্ণ PROPERTY

Property	কাজ
<code>name</code>	Error-এর নাম
<code>message</code>	Error-এর description
<code>stack</code>	Error কোথা থেকে এসেছে তার trace

PART 02 — EXCEPTION HANDLING: TRY, CATCH, FINALLY & THROW (14.11 – 14.26)

14.11 try ... catch

JavaScript-এ error handle করার সবচেয়ে গুরুত্বপূর্ণ system হলো:

14.11 JAVASCRIPT

RUN

```
try {  
    // risky code  
} catch (error) {  
    // error handling  
}
```

💡 EXAMPLE

14.11 JAVASCRIPT

RUN

```
try {  
    console.log(userName);  
} catch (error) {  
    console.log("Something went wrong!");  
}
```

TEXT

Something went wrong!

এখানে program পুরোপুরি crash না করে `catch` block-এ চলে গেছে।

14.12

catch

-এর Error Object

আমরা error সম্পর্কে information দেখতে পারি।

14.12 JAVASCRIPT

RUN

```
try {  
  console.log(userName);  
} catch (error) {  
  console.log(error.name);  
  console.log(error.message);  
}
```

```
ReferenceError  
userName is not defined
```

14.13

try ... catch

কীভাবে কাজ করে?

ধরো:

14.13 JAVASCRIPT

RUN

```
try {  
  console.log("Step 1");  
  
  console.log(userName);  
  
  console.log("Step 3");  
} catch (error) {  
  console.log("Error caught!");  
}
```

```
Step 1  
Error caught!
```

Step 3 execute হবে না।

কারণ error হওয়ার পর JavaScript সরাসরি `catch` block-এ চলে যায়।

14.14

finally

`finally` block **error হোক বা না হোক**, সাধারণত execute হয়।

Syntax:

14.14 JAVASCRIPT

RUN

```
try {  
  
} catch (error) {  
  
} finally {  
  
}
```

Example:

14.14 JAVASCRIPT

RUN

```
try {  
    console.log("Trying ... ");  
} catch (error) {  
    console.log("Error!");  
} finally {  
    console.log("Finished!");  
}
```

```
Trying...  
Finished!
```

Error হলে:

14.14 JAVASCRIPT

RUN

```
try {  
    console.log(userName);  
} catch (error) {  
    console.log("Error!");  
} finally {  
    console.log("Finished!");  
}
```

```
Error!  
Finished!
```

14.15 Real-Life Example — Database Connection

ধরো application database-এর সাথে connection করছে।

14.15 JAVASCRIPT

RUN

```
try {  
    console.log("Connecting to database ... ");  
  
    throw new Error("Database connection failed");  
  
} catch (error) {  
    console.log("Database error:", error.message);  
  
} finally {  
    console.log("Connection process finished.");  
}
```

```
Connecting to database...  
Database error: Database connection failed  
Connection process finished.
```

14.16**throw**

আমরা নিজেও manually error তৈরি করতে পারি।

14.16 JAVASCRIPT

RUN

```
throw new Error("Invalid data");
```

```
Error: Invalid data
```

14.17**throw**

+

try ... catch

14.17 JAVASCRIPT

RUN

```
try {  
  throw new Error("Something went wrong");  
} catch (error) {  
  console.log(error.message);  
}
```

TEXT

```
Something went wrong
```

14.18

User Input Validation

throw

বাস্তবে validation-এর সময় অনেক usefull

14.18 JAVASCRIPT

RUN

```
function registerUser(username) {  
  
  if (!username) {  
    throw new Error("Username is required");  
  }  
  
  console.log("User registered:", username);  
}  
  
try {  
  registerUser("");  
} catch (error) {  
  console.log(error.message);  
}
```

TEXT

```
Username is required
```

14.19 Real-Life Login Example

14.19 JAVASCRIPT

RUN

```
function login(username, password) {  
  
    if (!username) {  
        throw new Error("Username is required");  
    }  
  
    if (!password) {  
        throw new Error("Password is required");  
    }  
  
    console.log("Login successful");  
}  
  
try {  
    login("shariar", "");  
} catch (error) {  
    console.log("Login failed:", error.message);  
}
```

TEXT

Login failed: Password is required

14.20 Custom Error

নিজের application-এর জন্য custom error class তৈরি করা যায়।

14.20 JAVASCRIPT

RUN

```
class ValidationError extends Error {  
  
    constructor(message) {  
        super(message);  
  
        this.name = "ValidationError";  
    }  
  
}
```

এখন:

14.20 JAVASCRIPT

RUN

```
throw new ValidationError("Email is invalid");
```

ValidationError: Email is invalid

14.21 Custom Error ব্যবহার

14.21 JAVASCRIPT

RUN

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

function register(email) {

  if (!email.includes("@")) {
    throw new ValidationError("Invalid email");
  }

  console.log("Registration successful");
}

try {
  register("hello.com");
} catch (error) {
  console.log(error.name);
  console.log(error.message);
}
```

```
ValidationError
Invalid email
```

14.22 instanceof দিয়ে Error Check

Chapter 11-এ আমরা `instanceof` দেখেছি।

এখানেও ব্যবহার করা যায়।

14.22 JAVASCRIPT

RUN

```
try {
  throw new TypeError("Wrong type");
} catch (error) {

  if (error instanceof TypeError) {
    console.log("Type error detected");
  }
}
```

```
Type error detected
```

14.23 Multiple Error Handling

একাধিক ধরনের error আলাদাভাবে handle করা যায়।

14.23 JAVASCRIPT

RUN

```
try {  
  
  let user = null;  
  
  console.log(user.name);  
  
} catch (error) {  
  
  if (error instanceof TypeError) {  
    console.log("Type problem!");  
  
  } else if (error instanceof ReferenceError) {  
    console.log("Reference problem!");  
  
  } else {  
    console.log("Unknown error!");  
  }  
  
}
```

Type problem!

14.24 Optional Chaining দিয়ে Error এড়ানো

Chapter 3-এ আমরা [?.](#) দেখেছি।

এটা অনেক ক্ষেত্রে `TypeError` এড়াতে সাহায্য করে।

14.24 JAVASCRIPT

RUN

```
let user = null;  
  
console.log(user?.name);
```

TEXT

undefined

যেখানে:

14.24 JAVASCRIPT

RUN

```
console.log(user.name);
```

এটি `TypeError` দিতে পারে।

✦ কিন্তু মনে রাখবে

[?.](#) error handling-এর complete replacement নয়।

এটি শুধু নির্দিষ্ট null/undefined access safely handle করে।

14.25 Nullish Coalescing

?? দিয়ে fallback value দেওয়া যায়।

14.25 JAVASCRIPT

RUN

```
let username = null;

let displayName = username ?? "Guest";

console.log(displayName);
```

TEXT

Guest

এটাও application crash এড়ানোর ক্ষেত্রে usefull

14.26 finally কখন বেশি ব্যবহার হয়?

finally সাধারণত cleanup operation-এর জন্য usefull

যেমন:

- Database connection cleanup
- File cleanup
- Loading state বন্ধ করা
- Resource release

Example:

14.26 JAVASCRIPT

RUN

```
let loading = true;

try {
  console.log("Fetching data ...");
} catch (error) {
  console.log("Error!");
} finally {
  loading = false;
  console.log("Loading finished");
}
```

TEXT

Fetching data ...
Loading finished

PART 03 — MODERN DEBUGGING & CONSOLE API (14.27 – 14.37)

14.27 Debugging কী?

Debugging হলো program-এর problem খুঁজে বের করে ঠিক করার process।

Real life:

গাড়ি start হচ্ছে না → mechanic কারণ খুঁজছে → problem identify → repair

Programming:

Website কাজ করছে না → developer code inspect করছে → bug identify → fix

14.28 `console.log()` দিয়ে Debugging

সবচেয়ে basic debugging technique:

14.28 JAVASCRIPT

RUN

```
let price = 100;
let quantity = 3;

console.log(price);
console.log(quantity);

let total = price * quantity;

console.log(total);
```

TEXT

```
100
3
300
```

এভাবে বুঝতে পারবে কোন stage-এ value ভুল হচ্ছে।

14.29 Better `console.log()`

এভাবে:

14.29 JAVASCRIPT

RUN

```
let price = 100;
let quantity = 3;

console.log("Price:", price);
console.log("Quantity:", quantity);

let total = price * quantity;

console.log("Total:", total);
```

TEXT

```
Price: 100
Quantity: 3
Total: 300
```

এটা debugging-এর জন্য অনেক পরিষ্কার।

14.30 `console.error()`

Error message দেখাতে:

14.30 JAVASCRIPT

RUN

```
console.error("Database connection failed");
```

Browser console-এ এটি error হিসেবে দেখাবে।

14.31 `console.warn()`

Warning:

14.31 JAVASCRIPT

RUN

```
console.warn("Password is weak");
```

এটি developer-কে warning দেয়।

14.32

console.table()

Array of objects debug করতে এটি অসাধারণ usefull

14.32 JAVASCRIPT

RUN

```
let students = [
  {
    name: "Shariar",
    age: 23,
    department: "CSE"
  },
  {
    name: "Rahim",
    age: 22,
    department: "CSE"
  }
];

console.table(students);
```

Browser console-এ table format-এ দেখাবে।

এটি বড় data debug করার সময় খুব usefull

14.33

console.dir()

Object-এর structure inspect করতে:

14.33 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  skills: ["HTML", "CSS", "JavaScript"]
};

console.dir(user);
```

Browser DevTools-এ object structure expand করে দেখতে পারবে।

14.34

console.time()

এবং

console.timeEnd()

কোনো code কত সময় নিচ্ছে তা measure করা যায়।

14.34 JAVASCRIPT

RUN

```
console.time("Loop");

for (let i = 0; i < 1000000; i++) {
  // work
}

console.timeEnd("Loop");
```

Output example:

TEXT

Loop: 5.123ms

সময় machine/browser অনুযায়ী পরিবর্তিত হবে।

14.35

console.count()

কোনো code কতবার execute হচ্ছে দেখাতে:

14.35 JAVASCRIPT

RUN

```
function test() {  
  console.count("test function");  
}
```

```
test();  
test();  
test();
```

TEXT

```
test function: 1  
test function: 2  
test function: 3
```

Debugging-এর জন্য usefull

14.36

console.assert()

কোনো condition false হলে message দেখাবে।

14.36 JAVASCRIPT

RUN

```
let age = 15;  
  
console.assert(age ≥ 18, "User must be 18 or older");
```

TEXT

```
Assertion failed: User must be 18 or older
```

যদি:

14.36 JAVASCRIPT

RUN

```
let age = 20;  
  
console.assert(age ≥ 18, "User must be 18 or older");
```

তাহলে সাধারণত কোনো assertion message দেখাবে না।

14.37

console.trace()

কোন function call chain থেকে code এসেছে তা দেখতে:

14.37 JAVASCRIPT

RUN

```
function first() {  
  second();  
}  
  
function second() {  
  console.trace();  
}  
  
first();
```

Console-এ call stack দেখা যাবে।

এটি complex application debugging-এ usefull

14.38 Browser DevTools

JavaScript developer হিসেবে Browser DevTools অবশ্যই জানতে হবে।

Chrome-এ:

```
Right Click
→ Inspect
→ Console
```

অথবা:

```
TEXT
F12
```

14.39 DevTools-এর গুরুত্বপূর্ণ অংশ

Tool	কাজ
Console	Error/output/debug
Sources	JavaScript files
Network	API/request
Elements	HTML/CSS
Application	Storage/Cookies
Performance	Performance analysis
Memory	Memory analysis

14.40 Breakpoint

ধরো:

```
14.40 JAVASCRIPT RUN
function calculate() {
    let price = 100;
    let quantity = 3;

    let total = price * quantity;

    console.log(total);
}

calculate();
```

Sources panel-এ `let total = ...` লাইনে breakpoint বসালে JavaScript ওই line-এ execution pause করতে পারে।

তারপর তুমি variable-এর value দেখতে পারবে।

14.41

debugger

Code-এর মধ্যেই breakpoint দিতে পারো:

14.41 JAVASCRIPT

RUN

```
function calculate() {  
  
  let price = 100;  
  let quantity = 3;  
  
  debugger;  
  
  let total = price * quantity;  
  
  console.log(total);  
}  
  
calculate();
```

Browser DevTools খোলা থাকলে `debugger` line-এ execution pause করতে পারে।

14.42 Call Stack

ধরো:

14.42 JAVASCRIPT

RUN

```
function one() {  
  two();  
}  
  
function two() {  
  three();  
}  
  
function three() {  
  console.log("Hello");  
}  
  
one();
```

Call sequence:

```
one()  
↓  
two()  
↓  
three()
```

এটাকে **Call Stack** দিয়ে track করা যায়।

Error হলে stack trace দেখে বোঝা যায় কোন function chain-এর মাধ্যমে error এসেছে।

14.43 Scope Debugging

ধরো:

14.43 JAVASCRIPT

RUN

```
let globalValue = 100;

function test() {

    let localValue = 50;

    console.log(globalValue);
    console.log(localValue);
}

test();
```

Debugger-এর Scope section-এ দেখতে পারবে:

TEXT

Local Scope
global Scope

এতে variable কোথা থেকে এসেছে বোঝা সহজ হয়।

14.44 Defensive Programming

Defensive Programming মানে হলো আগে থেকেই সম্ভাব্য ভুল situation চিন্তা করে code লেখা।

খারাপ:

14.44 JAVASCRIPT

RUN

```
function calculateTotal(price, quantity) {
  return price * quantity;
}
```

যদি:

14.44 JAVASCRIPT

RUN

```
calculateTotal();
```

তাহলে result:

TEXT

NaN

আরও defensive:

14.44 JAVASCRIPT

RUN

```
function calculateTotal(price, quantity) {

  if (typeof price !== "number") {
    throw new TypeError("Price must be a number");
  }

  if (typeof quantity !== "number") {
    throw new TypeError("Quantity must be a number");
  }

  return price * quantity;
}
```

এখন:

14.44 JAVASCRIPT

RUN

```
try {
  console.log(calculateTotal("100", 2));
} catch (error) {
  console.log(error.message);
}
```

TEXT

Price must be a number

14.45 Real Project Example — API Response

ধরো backend থেকে user data আসছে।

14.45 JAVASCRIPT

RUN

```
let response = {
  user: {
    name: "Shariar"
  }
};

console.log(response.user.name);
```

কিন্তু কোনো সময় backend response হতে পারে:

14.45 JAVASCRIPT

RUN

```
let response = {};
```

তখন:

14.45 JAVASCRIPT

RUN

```
console.log(response.user.name);
```

সমস্যা হতে পারে।

Safe:

14.45 JAVASCRIPT

RUN

```
console.log(response.user?.name);
```

TEXT

undefined

14.46 API Error Handling – এর Basic Pattern

বাস্তবে asynchronous API handling আমরা পরে Promise/ `async/await` chapter-এ বিস্তারিত করব।

এখন basic idea:

14.46 JAVASCRIPT

RUN

```
try {
  let data = getData();

  console.log(data);
} catch (error) {
  console.error("Failed:", error.message);
}
```

14.47

Error.cause

একটি error-এর original cause অন্য error হিসেবে attach করা যায়।

14.47 JAVASCRIPT

RUN

```
try {  
  
    throw new Error("Database unavailable");  
  
} catch (error) {  
  
    throw new Error("Failed to load user data", {  
        cause: error  
    });  
  
}
```

এখানে outer error-এর:

14.47 JAVASCRIPT

RUN

```
error.cause
```

এর মধ্যে original error থাকতে পারে।

এটি বড় application-এর error chain বুঝতে usefull।

14.48

AggregateError

একাধিক error একসাথে represent করার জন্য `AggregateError` ব্যবহার করা যায়।

14.48 JAVASCRIPT

RUN

```
let errors = [  
    new Error("Database failed"),  
    new Error("API failed"),  
    new Error("Cache failed")  
];  
  
let aggregate = new AggregateError(  
    errors,  
    "Multiple operations failed"  
);  
  
console.log(aggregate);
```

এর মধ্যে multiple errors থাকবে।

14.49 Error Handling-এর Real-Life E-Commerce Example

ধরো আমরা product order করছি।

14.49 JAVASCRIPT

RUN

```
function placeOrder(product, quantity) {  
  
  if (!product) {  
    throw new Error("Product is required");  
  }  
  
  if (quantity <= 0) {  
    throw new Error("Quantity must be greater than 0");  
  }  
  
  return `Order placed: ${product} x ${quantity}`;  
}
```

ব্যবহার:

14.49 JAVASCRIPT

RUN

```
try {  
  
  let result = placeOrder("Laptop", 2);  
  
  console.log(result);  
  
} catch (error) {  
  
  console.error("Order failed:", error.message);  
  
}
```

TEXT

Order placed: Laptop x 2

ভুল input:

14.49 JAVASCRIPT

RUN

```
try {  
  
  let result = placeOrder("Laptop", 0);  
  
  console.log(result);  
  
} catch (error) {  
  
  console.error("Order failed:", error.message);  
  
}
```

TEXT

Order failed: Quantity must be greater than 0

14.50 কখন `try ... catch` ব্যবহার করবে?

সব code-এর চারপাশে `try ... catch` দেওয়া উচিত নয়।

বিশেষ করে যেখানে failure হওয়ার সম্ভাবনা আছে সেখানে ব্যবহার করা ভালো।

যেমন:

```
API request
↓
JSON parsing
↓
File operation
↓
User input validation
↓
Database operation
↓
External service
```

14.51 Bad Error Handling

এভাবে করা ঠিক নয়:

14.51 JAVASCRIPT

RUN

```
try {
    riskyFunction();
} catch (error) {
}
```

কারণ error silently ignore হয়ে যাচ্ছে।

এটা debugging কঠিন করে দেয়।

Better:

14.51 JAVASCRIPT

RUN

```
try {
    riskyFunction();
} catch (error) {
    console.error("Operation failed:", error);
}
```

14.52 Error Hide না করে Properly Handle করা

খারাপ:

14.52 JAVASCRIPT

RUN

```
try {
    loginUser();
} catch (error) {
    console.log("Error");
}
```

ভালো:

14.52 JAVASCRIPT

RUN

```
try {
    loginUser();
} catch (error) {

    console.error("Login failed:", {
        name: error.name,
        message: error.message
    });

}
```

14.53 User-এর সামনে Technical Error দেখানো উচিত?

সবসময় না।

Developer console:

```
TypeError: Cannot read properties of undefined
```

User-এর জন্য:

```
Something went wrong. Please try again.
```

অর্থাত্:

```
Developer → detailed error  
User → friendly message
```

এটা professional application-এর গুরুত্বপূর্ণ practice।

14.54 Complete Mini Project — Student Registration

এখন সব concept একসাথে ব্যবহার করি।

14.54 JAVASCRIPT

RUN

```
class ValidationError extends Error {

  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

function registerStudent(name, age, email) {

  if (!name) {
    throw new ValidationError("Name is required");
  }

  if (typeof age !== "number") {
    throw new TypeError("Age must be a number");
  }

  if (age < 18) {
    throw new ValidationError("Student must be at least 18");
  }

  if (!email.includes("@")) {
    throw new ValidationError("Invalid email");
  }

  return {
    name,
    age,
    email
  };
}

try {

  let student = registerStudent(
    "Shariar",
    23,
    "shariar@example.com"
  );

  console.table(student);

} catch (error) {

  console.error(error.name);
  console.error(error.message);

}
```

TEXT

name	Shariar
age	23
email	shariar@example.com

MUST KNOW 🔥 14.55 Must Know

Chapter 14 থেকে অবশ্যই জানতে হবে:

Error

JAVASCRIPT

RUN

```
Error
SyntaxError
ReferenceError
TypeError
RangeError
URIError
AggregateError
```

Error Handling

JAVASCRIPT

RUN

```
try
catch
finally
throw
```

Error Object

JAVASCRIPT

RUN

```
error.name
error.message
error.stack
error.cause
```

Custom Error

JAVASCRIPT

RUN

```
class MyError extends Error {}
```

Debugging

JAVASCRIPT

RUN

```
console.log()
console.error()
console.warn()
console.table()
console.dir()
console.time()
console.timeEnd()
console.count()
console.assert()
console.trace()
debugger
```

DevTools

```
Console
Sources
Network
Elements
Application
Performance
Memory
```

Concept	Example
Error create	<code>new Error("Failed")</code>
Throw error	<code>throw new Error()</code>
Handle	<code>try ... catch</code>
Cleanup	<code>finally</code>
Error name	<code>error.name</code>
Error message	<code>error.message</code>
Stack	<code>error.stack</code>
Custom error	<code>class MyError extends Error</code>
Debug output	<code>console.log()</code>
Debug table	<code>console.table()</code>
Warning	<code>console.warn()</code>
Debug breakpoint	<code>debugger</code>
Type check	<code>error instanceof TypeError</code>

Practice 1

এমন একটি function তৈরি করো:

JAVASCRIPT

RUN

```
divide(a, b)
```

যেখানে `b = 0` হলে:

TEXT

```
Cannot divide by zero
```

error throw করবে।

Practice 2

একটি:

JAVASCRIPT

RUN

```
registerUser()
```

function তৈরি করো যেখানে:

- username required
- email required
- password minimum 8 characters

না হলে custom error throw করবে।

Practice 3

এই data:

JAVASCRIPT

RUN

```
let students = [  
  { name: "Rahim", marks: 80 },  
  { name: "Karim", marks: 75 },  
  { name: "Shariar", marks: 90 }  
];
```

`console.table()` দিয়ে দেখাও।

Practice 4

`console.time()` ব্যবহার করে 1 থেকে 1,000,000 পর্যন্ত loop-এর execution time বের করো।

Practice 5

একটি custom error তৈরি করো:

JAVASCRIPT

RUN

```
class InsufficientBalanceError extends Error
```

এবং একটি:

JAVASCRIPT

RUN

```
withdraw(balance, amount)
```

function তৈরি করো।

Balance-এর চেয়ে amount বেশি হলে error throw করবে।

SUMMARY **Chapter 14 Final Summary — Error Handling Architecture**

এই chapter-এর মূল flow মনে রাখো:

```
Problem
↓
Error
↓
Detect
↓
try
↓
catch
↓
Handle
↓
finally
↓
Cleanup
```

আর developer হিসেবে:

```
Bug
↓
Debug
↓
Console
↓
DevTools
↓
Breakpoint
↓
Inspect Variables
↓
Find Cause
↓
Fix
```

★ সবচেয়ে গুরুত্বপূর্ণ ধারণা

Error Handling = application-এর error properly handle করা।

Debugging = error/bug-এর আসল কারণ খুঁজে বের করা।

NEXT CHAPTER **Chapter 15 Preview — DOM (Document Object Model)**

এখন JavaScript-এর পরবর্তী গুরুত্বপূর্ণ ধাপ হলো **Chapter 15 — DOM (Document Object Model)**, যেখানে আমরা JavaScript দিয়ে HTML page-এর element **select, create, change, delete** এবং **dynamically manipulate** করা শিখব।